



Lab of Algorithm Design

Assignment 1

Linked list Insertion Sort Analysis

Wilkend AUGUSTE

Student ID: 411321344

March 03, 2026

1. Algorithmic Mechanics

Insertion Sort builds a sorted sequence by using each element into its correct position.

Let the input linked list be: $L = (X_1, X_2, X_3, \dots, X_n)$

A dummy node is used to simplify insertion at the head.

Initialization:

$$dummy.next = NULL$$

For each node P in the original list:

1. Traverse the sorted list
2. Find position where:

$$prev.val \leq P.val < next.val$$

3. Perform pointer redirection.

Insertion operation:

$$\begin{aligned} P.next &= prev.next \\ prev.next &= P \end{aligned}$$

Insertion cost:

$$O(1)$$

Search cost depends on the size of the sorted sublist.

2. Complexity Analysis

Worst Case

Worst case occurs when the input is strictly decreasing.

$$n, n-1, n-2, \dots, 1$$

For the i -th insertion:

$$C_i = i - 1$$

Total comparisons:

$$T(n) = \sum_{i=1}^n (i - 1)$$

$$T(n) = 0 + 1 + 2 + \dots + (n - 1)$$

Arithmetic series:

$$T(n) = \frac{n(n - 1)}{2}$$

Thus

$$T(n) \in O(n^2)$$

Best Case

If the input list is already sorted:

$$1, 2, 3, \dots, n$$

Each insertion requires one comparison.

$$T(n) = n$$

Thus

$$T(n) \in O(n)$$

Space Complexity

The algorithm uses a constant number of pointers: *dummy*, *curr*, *prev*, *next*

So the memory usage is independent of n .

$$S(n) \in O(1)$$

3. Algorithm Variant

For a doubly linked list with a maintained tail pointer.

Insertion rule: If

$$X.val \geq tail.val$$

append to the tail.

Time complexity: $O(1)$

Otherwise perform backward search:

$$tail \rightarrow tail.prev \rightarrow \dots$$

Strictly Decreasing Input

Input:

$$n, n - 1, n - 2, \dots, 1$$

Total comparisons:

$$T(n) = \sum_{i=1}^n (i - 1)$$
$$T(n) = \frac{n(n-1)}{2} \rightarrow O(n^2)$$
$$T(n) \in O(n^2)$$

4. Implementation

```
1  /* PROGRAMMING TASK - INSERTION SORT FOR LINKED LIST
2  */
3
4  ListNode* insertionSortList(ListNode* head) {
5      if (!head || !head->next) return head;
6
7      // Dummy node handles "head insertion" boundary cases
8      ListNode* dummy = new ListNode(0);
9      ListNode* curr = head;
10
11     while (curr) {
12         ListNode* prev = dummy;
13
14         // Scan for first node in sorted part with a larger value
15         while (prev->next && prev->next->val < curr->val) {
16             prev = prev->next;
17         }
18
19         // Pointer Redirection: Cut and re-link
20         ListNode* nextTemp = curr->next;
21         curr->next = prev->next;
22         prev->next = curr;
23
24         curr = nextTemp;
25     }
26     return dummy->next;
27 }
28
29 /**
30  * REAL-WORLD SCENARIO ANALYSIS :
31  * Despite O(n^2) worst-case complexity,
32  * this approach is preferred when:
33  * 1. Dataset size is small: Lower overhead than recursive sorts.
34  * 2. Input sequence is nearly sorted: Performance approaches O(n).
35  * 3. Constant auxiliary memory: O(1) space complexity is required.
36  * 4. Online processing: Handles data one-by-one as it arrives.
37  */
```